

OS161 Shell: C2 Project Report

Oliviero Marco (361264)
Prencipe Pasquale E. (364623)
Tenore Mirko (358543)

September 2026

Contents

1	Introduction	2
1.1	Project scope	2
1.2	OS161 system call interface and layered architecture	2
2	Data structures design	3
2.1	struct <code>openfile</code>	4
2.2	struct <code>proc</code> and <code>fileTable</code>	4
2.3	struct <code>_processTable</code>	5
2.4	Concurrency in system calls	5
3	File system calls	6
3.1	<code>sys_open</code>	6
3.2	<code>sys_close</code>	8
3.3	<code>sys_read</code>	9
3.4	<code>sys_write</code>	11
3.5	<code>sys_lseek</code>	13
3.6	<code>sys_dup2</code>	15
3.7	<code>sys___getcwd()</code>	17
3.8	<code>sys_chdir()</code>	18
4	Process system calls	19
4.1	<code>sys_getpid()</code>	19
4.2	<code>sys__exit()</code>	20
4.3	<code>sys_waitpid()</code>	21
4.4	<code>sys_fork()</code>	24
4.5	<code>sys_execv()</code>	26
5	Conclusions	32

1 Introduction

1.1 Project scope

The main objective of the project was to extend the OS161 operating system to support multiprogramming and the execution of user-level programs from disk. Initially, the system lacked the ability to run actual concurrent processes managed by an interactive command shell, and programs could only be launched directly from the kernel menu using `runprogram`. By implementing the complete suite of requested system calls for file and process management, we were able to transform OS161 from a basic kernel into a functional system capable of safely executing user applications in isolated virtual memory spaces.

The complete source code is available at: github.com/sdoublesm/os161-shell.

1.2 OS161 system call interface and layered architecture

OS161 adopts a layered architecture designed to provide a clear separation between user space and kernel space. To request services from the operating system, userland applications rely on the GNU C Library, that exposes the user-level system call interface through `user/include/unistd.h`. Calls to these C functions trigger hardware traps that transfer control to the kernel and save the CPU state in a `trapframe`. The dispatcher then retrieves the system call number and its arguments from `trapframe` and invokes the corresponding kernel-side implementation, whose declaration is defined in `kern/include/syscall.h`. A clear example can be observed in the `open` system call: while the user calls `int open(const char *filename, int flags)` in the user space, the syscall dispatcher routes it to `int sys_open(userptr_t pathname, int flags, mode_t mode, int32_t *retval)`, using `userptr_t` to explicitly mark and safely handle the memory originating from the user space. A simplified version of the mentioned dispatcher (contained into `kern/arch/mips/syscall/syscall.c`) is shown below:

```
1 void syscall(struct trapframe *tf) {
2     int callno = tf->tf_v0;
3     int32_t retval;
4     int err;
5
6     switch (callno) {
7         // ...
8         case SYS_read:
9             err = sys_read((int) tf->tf_a0, (userptr_t) tf
10                ->tf_a1, (size_t) tf->tf_a2, &retval);
11                 break;
12         case SYS_write:
```

```

13     err = sys_write((int) tf->tf_a0, (userptr_t)
14     tf->tf_a1, (size_t) tf->tf_a2, &retval);
15         break;
16
17     // ... other system calls
18 }
19
20     // trapframe update on return
21 }

```

The handler selected from the switch-case performs the requested operation and, for file-related calls, interacts with the Virtual File System (VFS), that provides an higher-level interface to the underlying components including caches, individual file systems, buffer cache, and device drivers. Our implementation focuses on providing a consistent and secure mechanism for managing file descriptors and open files in general while preserving this separation between user-level requests and kernel-level operations.

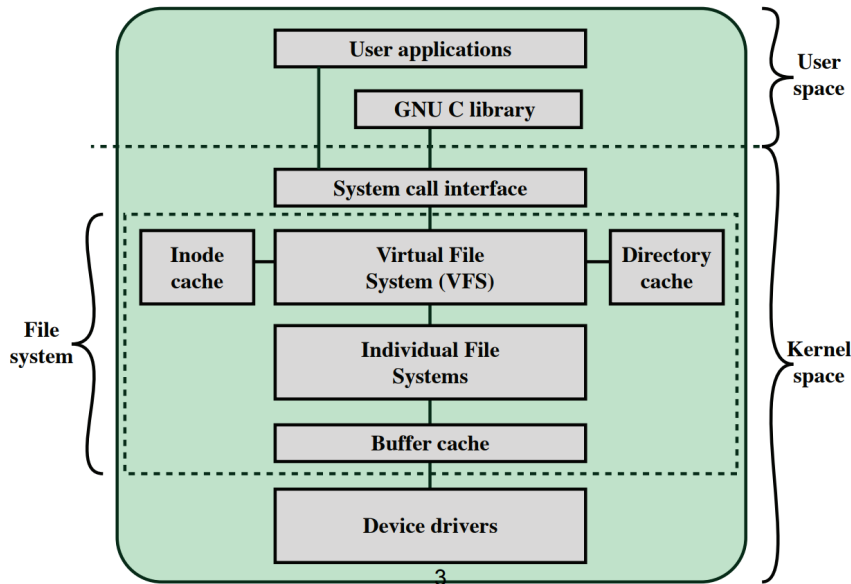


Figure 1: OS161 layered architecture.

2 Data structures design

The starting environment provided minimal support for running executables, lacking a robust process lifecycle abstraction and proper file descriptor tracking. To address these limitations, we designed our data structures by answering three

fundamental questions: how to represent opened files for each process, how to structure the processes and their informations, and how to manage all the active processes.

2.1 struct openfile

To answer the first question, we introduced the `struct openfile`, inside `kern/include/syscall.h`, in order to store details for every corresponding opened file. This structure contains a pointer to the `vnode` to locate the data, the open mode (read only, read and write, ..), the current file offset, a lock for concurrency protection (more below), and a reference count.

```
1 struct openfile{
2     struct vnode *vn;
3     int mode;
4     off_t offset;
5     struct lock *lk;
6     int ref_count;
7 };
```

2.2 struct proc and fileTable

Apart from the already present information about a process, other important ones had to be added, which are: the pid of the process and of its parent, a list of its children, the exit status and the flag `has_exited` to check if a process has already exited or not, the lock and the condition variable for synchronization, and, in order to keep track of opened files within every process, the `fileTable` array, that contains pointers to openfiles related to a certain `proc` structure, to ensure that every process manages its own independent set of files, while sharing the underlying `vnode` implementation. Indexes of this array serves as file descriptor (FD), allowing system calls like `read()`, `write()`, and `lseek()` to easily retrieve and operate on the correct opened file.

```
1 struct proc {
2     char *p_name;
3     struct spinlock p_lock;
4     unsigned p_numthreads;
5     struct addrspace *p_addrspace;
6     struct vnode *p_cwd;
7
8 #if OPT_SHELLPROJECT
9     pid_t p_id;
10    pid_t parent_id;
11    struct child_node *children_list;
12    int exit_status;
13    bool has_exited;
14    struct openfile *fileTable[OPEN_MAX];
```

```

15     struct cv *p_cv;
16     struct lock *p_lk;
17 #endif
18 };

```

2.3 struct _processTable

Since we want to achieve multiprogramming, we have to keep track of every active process. To do that, the global struct `_processTable` is useful, having an array of processes, a spinlock to achieve synchronization, a field to check if the table is active or not, and the index of the last process assigned to the array. The indexes of the array are also used as pids for the process, in this way, when we want to get a process by its pid, we simply access the array with that pid. This table is instantiated at the very beginning of the system, when it boots, and the first process of the array is assigned to the kernel process.

```

1 static struct _processTable{
2     int active;
3     struct proc *proc[MAX_PROC+1];
4     int last_i;
5     struct spinlock lk;
6 }processTable;
7 };

```

2.4 Concurrency in system calls

Can two different user-level processes find themselves running a system call at the same time? YES. Since the kernel supports concurrent execution, any shared kernel data structure accessed during a system call is highly vulnerable to race conditions. For instance, when a process calls `fork()`, the parent and child processes share the exact same `openfile` structures in memory. If both processes call `sys_read()` or `sys_write()` at the exact same time on the shared file descriptor, they could simultaneously read and modify the `offset` field, leading to inconsistent file positions and corrupted reads or writes. To safely support concurrent system calls, we explicitly inserted a `struct lock *lk` inside the `openfile` structure. By acquiring this lock before operations that modify the state, such as read, write, and lseek, and releasing it at the end, we ensure that updates to the shared file offset and accesses to the underlying `vnode` are consistent. This guarantees system stability and data integrity even when multiple processes are running system calls concurrently.

3 File system calls

3.1 sys_open

3.1.1 Code

```
1 int sys_open(userptr_t pathname, int flags, mode_t
   mode, int32_t *retval)
2 {
3     #if OPT_SHELLPROJECT
4         struct openfile *file;
5         struct vnode *vn;
6         int err;
7         struct lock *ftlock = curproc->p_lk;
8         int fd;
9
10        if (pathname==NULL){
11            return EFAULT;
12        }
13
14        // copy the string from user space to kernel space
15        // necessary to avoid direct access to user memory
16        char *kpath = (char *) kmalloc(PATH_MAX*sizeof(char)
17        );
18        size_t actual_len;
19        if (kpath==NULL){
20            return ENOMEM;
21        }
22
23        err = copyinstr(pathname, kpath, PATH_MAX, &
24        actual_len);
25        if (err) {
26            kfree(kpath);
27            return err;
28        }
29
30        err = vfs_open(kpath, flags, mode, &vn);
31        if (err) {
32            kfree(kpath);
33            return err;
34        }
35
36        file = kmalloc(sizeof(struct openfile));
37        if (file == NULL) {
38            vfs_close(vn);
39            kfree(kpath);
```

```

38     return ENOMEM;
39 }
40
41 // init openfile fields
42 file->vn = vn;
43 file->offset = 0;
44 file->mode = flags;
45 file->ref_count = 1;
46 file->lk = lock_create("filelock");
47 kfree(kpath);
48
49 if (file->lk == NULL) {
50     kfree(file);
51     vfs_close(vn);
52     return ENOMEM;
53 }
54
55 // insert openfile at the first free slot inside the
    filetable
56 lock_acquire(ftlock);
57 for (fd = 0; fd < OPEN_MAX; fd++) {
58     if (curproc->fileTable[fd] == NULL) {
59         curproc->fileTable[fd] = file;
60         break;
61     }
62 }
63 lock_release(ftlock);
64 // no free slot
65 if (fd == OPEN_MAX) {
66     lock_destroy(file->lk);
67     kfree(file);
68     vfs_close(vn);
69     return EMFILE; // per-process limit on the number
        of open file reached
70 }
71 *retval = fd;
72 return 0;
73 #else
74 (void)pathname;
75 (void)flags;
76 (void)mode;
77 (void)retval;
78 return ENOSYS;
79 #endif
80 }

```

3.1.2 Description

The `sys_open` system call initializes the interaction with a file, allocating the corresponding `openfile` structure and returning a file descriptor. After first safely copying the filename from user space to kernel space, it calls `vfs_open()` (which calls `VOP_READ` inside) to obtain the corresponding `vnode`. After initializing access mode, file offset and synchronization lock, it scans the calling process's (`curproc`) `fileTable` to find the first available file descriptor (`fd`), that is then mapped to the newly created `openfile` and returned to the user.

3.2 sys_close

3.2.1 Code

```
1 int sys_close(int fd){
2 #if OPT_SHELLPROJECT
3     struct openfile *file;
4     struct lock *ftlock = curproc->p_lk;
5     if (fd < 0 || fd >= OPEN_MAX) {
6         return EBADF;
7     }
8     lock_acquire(ftlock);
9     file = curproc->fileTable[fd];
10
11     if (file == NULL) {
12         lock_release(ftlock);
13         return EBADF;
14     }
15     // remove from filetable
16     curproc->fileTable[fd] = NULL;
17     lock_release(ftlock);
18     openfile_decref(file);
19     return 0;
20 #else
21     (void)fd;
22     return ENOSYS;
23 #endif
24 }
```

3.2.2 Description

The `sys_close()` system call releases a file descriptor previously obtained through `open()`. After validating that `fd` is within the valid range, we acquire the file table lock and retrieve the corresponding `openfile` structure. If no file is currently associated with `fd`, an `EBADF` error is returned.

Once the entry has been located, it is immediately removed from the file table, so that the descriptor can be reused by future calls to `open()`. The actual release of the underlying file resource is delegated to `openfile_decref()`, reported below:

```

1 #if OPT_SHELLPROJECT
2 static void openfile_decref(struct openfile *file)
3 {
4     lock_acquire(file->lk);
5     file->ref_count--;
6
7     if (file->ref_count > 0) {
8         // still in use by someone else -> do nothing
9         lock_release(file->lk);
10    } else {
11        // effectively close
12        lock_release(file->lk);
13        vfs_close(file->vn);
14        lock_destroy(file->lk);
15        kfree(file);
16    }
17 }
18 #endif

```

This function decrements the reference count of the given `openfile` structure. If other file descriptors, either in this process or in a child process obtained via `fork()`, still refer to the same `openfile`, the function simply releases the lock and returns, leaving the structure intact. Otherwise, once the reference count reaches zero, the underlying `vnode` is closed through `vfs_close()`, the lock is destroyed, and the structure itself is freed, since no descriptor refers to it anymore.

3.3 sys_read

3.3.1 Code

```

1 int sys_read(int fd, userptr_t buf, size_t size,
2             int32_t *retval)
3 {
4     #if OPT_SHELLPROJECT
5         struct openfile *file;
6         struct lock *ftlock = curproc->p_lk; // file table
7         lock-protected
8         struct iovec iov;
9
10        struct uio userio;
11        int err;

```

```

10  int amode;
11
12  // validity check file descriptor
13  if (fd < 0 || fd >= OPEN_MAX) {
14      return EBADF;
15  }
16
17  // get the openfile item from fileTable
18  lock_acquire(ftlock);
19  file = curproc->fileTable[fd];
20  lock_release(ftlock);
21
22  if (file == NULL) {
23      return EBADF;
24  }
25
26  // check mode is compatible with read
27  // file->mode in AND with ACCESS MODE (user
    permission)
28  amode = file->mode & O_ACCMODE;
29  if (amode == O_WRONLY) {
30      return EBADF;
31  }
32
33  // acquire lock on file offset
34  lock_acquire(file->lk);
35
36  /* set iovec and uio structs for the transfer toward
    the userspace */
37  // set iovec fields
38  iov.iov_ubase = buf;
39  iov.iov_len = size;
40
41  userio.uio_iov = &iov;
42  userio.uio_iovcnt = 1;
43
44  // don't force offset=0 for fd<=2!
45  userio.uio_offset = file->offset;
46
47  userio.uio_resid = size;
48  userio.uio_segflg = UIO_USERSPACE;
49  userio.uio_rw = UIO_READ;
50  userio.uio_space = proc_getas();
51
52  // ! use VOP_READ function from VFS
53  err = VOP_READ(file->vn, &userio);

```

```

54     if (err) {
55         lock_release(file->lk);
56         return err;
57     }
58
59     // offset update
60     file->offset = userio.uio_offset;
61
62     // retval is the number of read bytes
63     *retval = size - userio.uio_resid;
64     //lock release
65     lock_release(file->lk);
66     return 0;
67 #else
68     (void)fd;
69     (void)buf;
70     (void)size;
71     (void)retval;
72     return ENOSYS;
73 #endif
74 }

```

3.3.2 Description

The `sys_read` system call handles data reading from an open file into a user-space buffer. After checking the validity of the file descriptor, it retrieves the corresponding `openfile` from the calling process's (`curproc`) `fileTable` and verifies the access permissions. After acquiring the file lock, it sets up `iovec` and `uio` structures to manage a safe data transfer toward user space. Finally, it calls the VFS function `VOP_READ()` to perform the actual read operation, updates the file offset, and returns the number of bytes read.

3.4 `sys_write`

3.4.1 Code

```

1  int sys_write(int fd, userptr_t buf, size_t size,
2              int32_t *retval)
3  {
4      #if OPT_SHELLPROJECT
5          struct openfile *file;
6          struct lock *ftlock = curproc->p_lk;
7          struct iovec iov;
8          struct uio userio;
9          int err;
10         int amode;

```

```

10
11     if (fd < 0 || fd >= OPEN_MAX) {
12         return EBADF;
13     }
14
15     lock_acquire(ftlock);
16     file = curproc->fileTable[fd];
17     lock_release(ftlock);
18     if (file == NULL) {
19         return EBADF;
20     }
21
22     // check file mode is not readonly
23     amode = file->mode & O_ACCMODE;
24     if (amode == O_RDONLY) {
25         return EBADF;
26     }
27
28     lock_acquire(file->lk);
29
30     iov.iov_ubase = buf;
31     iov.iov_len = size;
32
33     userio.uio_iov = &iov;
34     userio.uio_iovcnt = 1;
35
36     userio.uio_offset = file->offset;
37
38     userio.uio_resid = size;
39     userio.uio_segflg = UIO_USERSPACE;
40     userio.uio_rw = UIO_WRITE;
41     userio.uio_space = proc_getas();
42
43     // ! use VOP_WRITE function from VFS
44     err = VOP_WRITE(file->vn, &userio);
45     if (err) {
46         lock_release(file->lk);
47         return err;
48     }
49
50     file->offset = userio.uio_offset;
51     *retval = size - userio.uio_resid;
52
53     lock_release(file->lk);
54     return 0;
55 #else

```

```

56     (void)fd;
57     (void)buf;
58     (void)size;
59     (void)retval;
60     return ENOSYS;
61 #endif
62 }

```

3.4.2 Description

The `sys_write` system call manages writing data from a user-space buffer to an open file. Similar to the read operation, it validates the file descriptor, fetches the `openfile` structure from the `fileTable`, and checks the access mode. After acquiring the file's lock, it initializes `iovec` and `uio` structures for data transfer from user space. It then invokes `VOP_WRITE()` to write the data, updates the offset in the `openfile`, and returns the amount of successfully written bytes.

3.5 `sys_lseek`

3.5.1 Code

```

1  int sys_lseek(int fd, off_t offset, int whence,
2               int64_t *retval){
3  #if OPT_SHELLPROJECT
4      struct openfile *file = NULL;
5      struct lock *ftlock = curproc->p_lk;
6      off_t new_offset;
7      struct stat stats;
8      int err = 0;
9
10     if (fd < 0 || fd >= OPEN_MAX) {
11         return EBADF;
12     }
13
14     lock_acquire(ftlock);
15     file = curproc->fileTable[fd];
16     lock_release(ftlock);
17     if (file == NULL) {
18         return EBADF;
19     }
20
21     // check: file is seekable?
22     if (!VOP_ISSEEKABLE(file->vn)) {
23         return ESPIPE; // not seekable
24     }
25
26     // lock per modifica offset

```

```

25 lock_acquire(file->lk);
26
27 switch (whence) {
28     case SEEK_SET:
29         new_offset = offset;
30         break;
31     case SEEK_CUR:
32         new_offset = file->offset + offset;
33         break;
34     case SEEK_END:
35         // ! VOP_STAT fills the struct stats which
36         // contains the length of the file
37         err = VOP_STAT(file->vn, &stats); // return 0 on
38         // success
39         if (err) { // error
40             lock_release(file->lk);
41             return err;
42         }
43         new_offset = stats.st_size + offset;
44         break;
45     default:
46         lock_release(file->lk);
47         return EINVAL;
48 }
49
50 // update offset
51 if (new_offset < 0) {
52     lock_release(file->lk);
53     return EINVAL;
54 }
55 file->offset = new_offset;
56 *retval = new_offset;
57 lock_release(file->lk);
58
59 return 0; // success
60 #else
61 (void)fd;
62 (void)offset;
63 (void)whence;
64 (void)retval;
65 return ENOSYS;
66 #endif
67 }

```

3.5.2 Description

The `sys_lseek` system call changes the current offset of an open file. At first, it retrieves the `openfile` from the `fileTable` and checks if the underlying file is seekable, and after acquiring the file's lock, it computes the new offset based on the `whence` directive (`SEEK_SET`, `SEEK_CUR`, or `SEEK_END`) and updates the `openfile` structure accordingly. Finally, it returns the newly computed offset to the user.

3.6 `sys_dup2`

3.6.1 Code

```
1 int sys_dup2(int oldfd, int newfd, int32_t *retval){
2 #if OPT_SHELLPROJECT
3     struct openfile *file;
4     struct openfile *to_close;
5     struct lock *ftlock = curproc->p_lk;
6
7     // both fds must stay in a valid range
8     if (oldfd < 0 || oldfd >= OPEN_MAX ||
9         newfd < 0 || newfd >= OPEN_MAX) {
10         return EBADF;
11     }
12
13     lock_acquire(ftlock);
14
15     file = curproc->fileTable[oldfd];
16     if (file == NULL) {
17         lock_release(ftlock);
18         return EBADF;
19     }
20
21     // dup2 of a fd on itself
22     if (oldfd == newfd) {
23         lock_release(ftlock);
24         *retval = newfd;
25         return 0;
26     }
27
28     to_close = curproc->fileTable[newfd];
29
30     // oldfd and newfd already share the same openfile
31     if (file == to_close) {
32         lock_release(ftlock);
33         *retval = newfd;
```

```

34     return 0;
35 }
36
37     // we add the ref before installing it in newfd,
38     // by doing this
39     // we avoid situations where newfd is never
40     // visible, like "closed but not reassigned"
41     lock_acquire(file->lk);
42     file->ref_count++;
43     lock_release(file->lk);
44
45     curproc->fileTable[newfd] = file;
46
47     lock_release(ftlock);
48
49     // if newfd was open on another file, we close it
50     if (to_close != NULL) {
51         openfile_decref(to_close);
52     }
53
54     *retval = newfd;
55     return 0;
56 #else
57     (void)oldfd;
58     (void)newfd;
59     (void)retval;
60     return ENOSYS;
61 #endif
62 }

```

3.6.2 Description

The `sys_dup2()` system call duplicates an existing file descriptor `oldfd` onto `newfd`, so that both descriptors refer to the same open file. Before proceeding, we check that both `oldfd` and `newfd` are within the valid range: if not, an `EBADF` error is returned.

We then retrieve the `openfile` structure associated with `oldfd`. If `oldfd` is not currently open, an `EBADF` error is returned. Two special cases are handled early: if `oldfd` and `newfd` are the same, or if they already point to the same `openfile` structure, the function simply returns `newfd` without performing any further operation, since duplicating a descriptor onto itself would be redundant.

In the general case, the reference count of the `openfile` associated with `oldfd` is incremented before it is installed in `newfd`. This ordering is important, since incrementing the reference count first avoids situations in which `newfd` would temporarily point to nothing, for example if the old entry at `newfd` were closed

before the new one was assigned. Once the new entry has been installed, if newfd was previously associated with another open file, that file is closed through `openfile_decref()`, which decrements its reference count and releases the underlying resources once no descriptor refers to it anymore.

Finally, the function returns newfd, confirming that the duplication was successful.

3.7 sys___getcwd()

3.7.1 Code

```
1 int sys___getcwd(userptr_t buf, size_t buflen, int32_t
   *retval)
2 {
3     #if OPT_SHELLPROJECT
4         struct iovec iov;
5         struct uio userio;
6         int result;
7
8         // uio for transferring into user buffer
9         iov.iov_ubase = buf;
10        iov.iov_len = buflen;
11
12        userio.uio_iov = &iov;
13        userio.uio_iovcnt = 1;
14        userio.uio_offset = 0;
15        userio.uio_resid = buflen;
16        userio.uio_segflg = UIO_USERSPACE;
17        userio.uio_rw = UIO_READ;
18        userio.uio_space = proc_getas();
19
20        result = vfs_getcwd(&userio);
21        if (result) {
22            return result;
23        }
24
25        // bytes written into user buffer
26        *retval = buflen - userio.uio_resid;
27        return 0;
28    #else
29        (void)buf;
30        (void)buflen;
31        (void)retval;
32        return ENOSYS;
33    #endif
34 }
```

3.7.2 Description

The `sys__getcwd()` system call allows a process to retrieve the absolute path of its current working directory into a user-space buffer. To perform the transfer, we set up an `iovec` and a `uio` structure pointing to the user buffer `buf`, with a length of `buflen` bytes.

The `uio_rw` field is set to `UIO_READ`, since the operation copies data from the kernel into the user buffer, and `uio_segflg` is set to `UIO_USERSPACE`, indicating that the destination buffer resides in user space. The address space of the current process is retrieved through `proc_getas()` and assigned to `uio_space`, so that the transfer can be correctly validated against the process's address space.

The actual retrieval of the current directory is delegated to `vfs_getcwd()`, which fills the buffer through the `uio` structure by interacting with the VFS layer. If the call fails, the corresponding error is returned.

Finally, the number of bytes actually written into the user buffer is computed as the difference between `buflen` and `uio_resid`, the latter representing the number of bytes that were not transferred, and stored in `retval`.

3.8 `sys_chdir()`

3.8.1 Code

```
1 int sys_chdir(const_userptr_t pathname)
2 {
3     #if OPT_SHELLPROJECT
4         char *path;
5         int result;
6
7         if (pathname == NULL) {
8             return EFAULT;
9         }
10
11         path = kmalloc(PATH_MAX);
12         if (path == NULL) {
13             return ENOMEM;
14         }
15
16         // safe copy from user space
17         result = copyinstr(pathname, path, PATH_MAX, NULL);
18         if (result) {
19             kfree(path);
20             return result;
21         }
22
23         result = vfs_chdir(path);
24         kfree(path);
```

```

25     return result;
26 #else
27     (void)pathname;
28     return ENOSYS;
29 #endif
30 }

```

3.8.2 Description

The `sys_chdir()` system call allows a process to change its current working directory to the one specified by `pathname`. Before proceeding, we check that the `pathname` pointer is not `NULL`, in which case an `EFAULT` error is returned. We then allocate a kernel buffer of size `PATH_MAX` to safely copy the path from user space, returning `ENOMEM` if the allocation fails.

The actual copy is performed by `copyinstr()`, which safely transfers the string from user space to kernel space; this step is necessary because the kernel cannot directly dereference user-space pointers. If the copy fails, the corresponding error is returned and the allocated buffer is freed.

Once the path has been safely copied into kernel space, the actual directory change is delegated to `vfs_chdir()`, which updates the current directory of the process by interacting with the VFS layer. The kernel buffer is then freed, and the result of `vfs_chdir()` is returned to the caller, whether it indicates success or an error.

4 Process system calls

4.1 `sys_getpid()`

4.1.1 Code

```

1 pid_t sys_getpid(void){
2 #if OPT_SHELLPROJECT
3     KASSERT(curproc != NULL);
4     return curproc->p_id;
5 #else
6     return ENOSYS;
7 #endif
8 }

```

4.1.2 Description

The `sys_getpid()` system call is the easiest one to implement, yet it is essential for an OS to work. The call simply returns the pid of the current process, which is useful for example for a process to self identify.

4.2 sys__exit()

4.2.1 Code

```
1 void sys__exit(int exitcode){
2 #if OPT_SHELLPROJECT
3     struct proc *p = curproc;
4     for (int i = 0; i < OPEN_MAX; i++){
5         if (curproc->fileTable[i] != NULL){
6             lock_acquire(curproc->fileTable[i]->lk);
7             curproc->fileTable[i]->ref_count--;
8
9             if (curproc->fileTable[i]->ref_count > 0)
10 lock_release(curproc->fileTable[i]->lk);
11         else{
12             lock_release(curproc->fileTable[i]->lk
13 );
14             vfs_close(curproc->fileTable[i]->vn);
15             lock_destroy(curproc->fileTable[i]->lk
16 );
17             kfree(curproc->fileTable[i]);
18         }
19         curproc->fileTable[i] = NULL;
20     }
21
22     struct addrspace *as = proc_setas(NULL);
23     as_deactivate();
24     if (as != NULL) as_destroy(as);
25
26     p->exit_status = exitcode & 0xff;
27     p->has_exited = true;
28
29     lock_acquire(p->p_lk);
30     cv_signal(p->p_cv, p->p_lk);
31     lock_release(p->p_lk);
32
33 #else
34     struct addrspace *as = proc_getas();
35     as_destroy(as);
36 #endif
37     thread_exit();
38     panic("thread_exit returned (should not happen)\n"
39 );
40     (void) exitcode;
41 }
```

4.2.2 Description

The `sys__exit()` system call is probably the most important one when it comes to memory leaks in OS. This is because, when a process ends, it has to free every field except for `exit_status` and, in our implementation, `has_exited`. This way, the memory is freed for reuse, and the exit status of the process can be collected by the parent via `waitpid()`.

The exit status is retrieved from the last 8 bits of exitcode. After retrieving the code, the process sends a signal on the condition variable to wake up the process that was waiting on it.

4.3 `sys_waitpid()`

4.3.1 Code

```
1 int sys_waitpid(pid_t pid, int *status, int options,
2   int32_t *retval){
3     #if OPT_SHELLPROJECT
4       KASSERT(curproc != NULL);
5
6       // cannot wait on itself
7       // if so, error ECHILD
8       if (pid == curproc->p_id) return ECHILD;
9
10      // cannot wait on a process which is not its child
11      // if so, error ECHILD
12      if (!proc_check_child(curproc, pid)) return ECHILD;
13
14      // the status pointer has to be addressed to a
15      // multiple of 4 address
16      // if not (so it is unaligned), it cannot contain
17      // an integer, so error EFAULT
18      if (status != NULL && ((vaddr_t) status % 4 != 0))
19        return EFAULT;
20
21      // let's find the process, if it doesn't exist,
22      // error ESRCH
23      struct proc *p = proc_search_pid(pid);
24      if (p == NULL) return ESRCH;
25
26      // different values for options. It should be 0,
27      // but some options could be implemented
28      // here for example WNOHANG is implemented
29      // if wrong value of options is given, error
30      EINVAL
```

```

24     switch (options){
25         case 0:
26             break;
27         case WNOHANG:
28             if (!(p->has_exited)){
29                 *retval = 0;
30                 return 0;
31             }
32             break;
33         default:
34             return EINVAL;
35             break;
36     }
37
38     int s;
39
40     // let's wait the process exit
41     s = proc_wait(p);
42     if (status != NULL){
43         s = _MKWAIT_EXIT(s);
44         int err = copyout(&s, (userptr_t) status,
45 sizeof(int));
46         if (err){
47             return err;          // err should be
48 automatically EFAULT if it was an invalid pointer
49         };
50     }
51     *retval = p->p_id;
52
53     proc_destroy(p);
54
55     return 0;
56 #else
57     (void) options;
58     (void) pid;
59     (void) status;
60     (void) retval;
61     return ENOSYS;
62 #endif
63 }

```

4.3.2 Description

The `sys_waitpid()` system call is used by a process to wait on and retrieve the exit code of one of its children. It returns the pid of the child awaited and save

the status in a variable passed by reference to the function. Before proceeding, we must check the possibility of some errors. First a process cannot wait on itself or on a process which is not in his children list, resulting in an error `ECHILD`. The status must be a valid aligned pointer in the userspace or it would be an error `EFAULT`. If the options value corresponds to a wrong value (for example an option not implemented), it will result in an error `EINVAL`. Finally, if the process to wait on doesn't exist, error `ESRCH`.

If the function passes all the controls, then the current process will wait on the target process. If the process to wait on has already exited (an information obtained thanks to `has_exited`), the function returns immediately. In the case of option selected is `WNOHANG`, if the process to wait on has not exited yet, the `waitpid` returns 0. In the manual it is specified that status could be `NULL`, in this case we don't want to save the exit status.

The wait itself is done by the function `proc_wait()`, reported below:

```

1 int proc_wait(struct proc *proc){
2 #if OPT_SHELLPROJECT
3     int return_status;
4     KASSERT(proc != NULL);
5     KASSERT(proc != kproc);
6     lock_acquire(proc->p_lk);
7     while (!proc->has_exited) cv_wait(proc->p_cv, proc->
8         p_lk);
9     return_status = proc->exit_status;
10    lock_release(proc->p_lk);
11
12    bool threads_left = true;
13    while (threads_left){
14        spinlock_acquire(&proc->p_lock);
15        if (proc->p_numthreads == 0) threads_left = false;
16        spinlock_release(&proc->p_lock);
17        if (threads_left) thread_yield();
18    }
19    return return_status;
20 #else
21     (void) proc;
22     return -1;
23 #endif
24 }
```

First, the process waits on the condition variable until the target process ends (if the target process has already exited, the wait never starts). After the wait, the return status is saved and, before returning, the function checks whether all threads of that process have terminated.

4.4 sys_fork()

4.4.1 Code

```
1 int sys_fork(struct trapframe *ctf, pid_t *retval){
2 #if OPT_SHELLPROJECT
3     struct trapframe *tf_child;
4     struct proc *cp;
5     int result;
6
7     KASSERT(curproc != NULL);
8
9     if (!proc_find_free_slot()) return ENPROC;
10
11     cp = proc_create_runprogram(curproc->p_name);
12     if (cp == NULL) return ENOMEM;
13
14     as_copy(curproc->p_addrspace, &(cp->p_addrspace));
15     if (cp->p_addrspace == NULL){
16         proc_destroy(cp);
17         return ENOMEM;
18     }
19
20     tf_child = kmalloc(sizeof(struct trapframe));
21     if (tf_child == NULL){
22         proc_destroy(cp);
23         return ENOMEM;
24     }
25     memcpy(tf_child, ctf, sizeof(struct trapframe));
26
27     // -- procedure for copying the file descriptors
28     lock_acquire(curproc->p_lk);
29     for (int i = 0; i < OPEN_MAX; i++) {
30         struct openfile *file = curproc->fileTable[i];
31         if (file != NULL) {
32             lock_acquire(file->lk);
33             file->ref_count++;
34             lock_release(file->lk);
35         }
36         cp->fileTable[i] = file;
37     }
38     lock_release(curproc->p_lk);
39     // ---
40     if (proc_insert_child_in_parent(curproc, cp->p_id)
41         == -1){
42         kfree(tf_child);
```



```

42     proc_destroy(cp);
43     return ENOMEM;
44 }
45 cp->parent_id = curproc->p_id;
46
47     result = thread_fork(curthread->t_name, cp,
48     call_enter_forked_process, (void *) tf_child, (
49     unsigned long) 0);
50     if (result){
51         proc_remove_child_from_parent(curproc, cp->
52         p_id);
53         proc_destroy(cp);
54         kfree(tf_child);
55         return ENOMEM;
56     }
57
58     *retval = cp->p_id;
59     return 0;
60 #else
61     (void) ctf;
62     (void) retval;
63     return ENOSYS;
64 #endif
65 }

```

4.4.2 Description

The `sys_fork()` system call is the most important one for multiprogramming, and its implementation had to be handled carefully. First, we check whether there is space in the process table for another process; if not, an `ENPROC` error is returned. After successfully creating a process ready to run, we copy the address space, the trapframe, and the file table of the current process, which will be the parent process. If there is no error, the newly created process is added to the children list of the parent, and the parent pid of the child is set to the current process's pid. Once everything has completed successfully, we run the new process by means of a thread fork, passing the name, the child process, the function to start from, and the trapframe. It is important to highlight that the function `call_enter_forked_process()` is just an auxiliary function that calls the important one: `enter_forked_process()`. This function is responsible for setting the return value to 0 (since this is the return value of the child after a fork), setting the error code to 0, incrementing the program counter, activating the address space, and finally switching the child process to user mode. Since `enter_forked_process()` does not return, the `sys_fork()` system call continues execution only in the parent process, returning the pid of the child process just created.

4.5 sys_execv()

4.5.1 Code

```
1 int sys_execv(const char *program, char **args){
2 #if OPT_SHELLPROJECT
3     struct vnode *v;
4     vaddr_t entrypoint, stackptr;
5     int result;
6
7     if (program == NULL || args == NULL) return EFAULT
8     ;
9
10    // safe copy of the program name from user space
11    to kernel space
12    char *kern_prog = (char *) kmalloc(PATH_MAX *
13    sizeof(char));
14    if (kern_prog == NULL){
15        return ENOMEM;
16    }
17    result = copyinstr((userptr_t) program, kern_prog,
18    PATH_MAX, NULL);
19    if (result){
20        kfree(kern_prog);
21        return result;
22    }
23
24    // safe copy of the args array from user space to
25    kernel space
26    userptr_t temp_ptr;
27    int argc = 0;
28    int total_bytes = 0;
29    // first we have to count exactly how many
30    arguments we have
31    while(1){
32        result = copyin((userptr_t)args + (argc*sizeof
33    (userptr_t)), &temp_ptr, sizeof(userptr_t));
34        if (result){
35            kfree(kern_prog);
36            return result;
37        }
38        if (temp_ptr == NULL) break;
39        argc++;
40    }
41    if (argc == 0){
42        kfree(kern_prog);
```

```

36         return EINVAL;
37     }
38
39     total_bytes += (argc+1) * sizeof(char *);
40
41     // now we can actually allocate space for the
42     arguments and copy them
43     char *arg_buffer = (char *) kcalloc(4096);
44     if (arg_buffer == NULL){
45         kfree(kern_prog);
46         return ENOMEM;
47     }
48
49     size_t *arg_offsets = (size_t *) kcalloc(argc *
50     sizeof(size_t));
51     if (arg_offsets == NULL){
52         kfree(arg_buffer);
53         kfree(kern_prog);
54         return ENOMEM;
55     }
56     size_t total_offset = 0;
57
58     for (int i = 0; i < argc; i++){
59         userptr_t str_ptr;
60         result = copyin((userptr_t) args + (i * sizeof
61         (userptr_t)), &str_ptr, sizeof(userptr_t));
62         if (result){
63             kfree(arg_offsets);
64             kfree(arg_buffer);
65             kfree(kern_prog);
66             return result;
67         }
68
69         size_t actual_length;
70         result = copyinstr(str_ptr, arg_buffer +
71         total_offset, 4096 - total_offset, &actual_length);
72         if (result){
73             kfree(arg_offsets);
74             kfree(arg_buffer);
75             kfree(kern_prog);
76             if (result == ENAMETOOLONG) return E2BIG;
77             return result;
78         }
79
80         arg_offsets[i] = total_offset;
81         total_offset += actual_length;

```

```

78     }
79
80     /* Open the file. */
81     result = vfs_open(kern_prog, O_RDONLY, 0, &v);
82     if (result) {
83         kfree(arg_offsets);
84         kfree(arg_buffer);
85         kfree(kern_prog);
86         return result;
87     }
88
89     struct addrspace *old_as;
90     struct addrspace *new_as;
91
92     // let's now create the new address space
93     new_as = as_create();
94     if (new_as == NULL){
95         vfs_close(v);
96         kfree(arg_offsets);
97         kfree(arg_buffer);
98         kfree(kern_prog);
99         return ENOMEM;
100     }
101
102     // we save the old address space for later
103     // destruction
104     old_as = proc_getas();
105
106     // we switch to the new as and activate it
107     proc_setas(new_as);
108     as_activate();
109
110     /* Load the executable. */
111     result = load_elf(v, &entrypoint);
112     if (result) {
113         /* p_addrspace will go away when curproc is
114         destroyed */
115         proc_setas(old_as);
116         as_activate();
117         as_destroy(new_as);
118         vfs_close(v);
119         kfree(arg_offsets);
120         kfree(arg_buffer);
121         kfree(kern_prog);
122         return result;
123     }

```

```

122     vfs_close(v);
123
124
125     // we create the user's stack in the new as
126     result = as_define_stack(new_as, &stackptr);
127     if (result) {
128         proc_setas(old_as);
129         as_activate();
130         as_destroy(new_as);
131         kfree(arg_offsets);
132         kfree(arg_buffer);
133         kfree(kern_prog);
134     return result;
135 }
136
137     // first we have to make space in the user stack
138     // enough to insert all the arguments
139     // we will save each of these address because
140     // later we will have to insert them too as pointers
141     // in the stack
142     // because they will be the values of the user's
143     argv
144     userptr_t *string_addr = kmalloc(argc * sizeof(
145     userptr_t));
146     if (string_addr == NULL){
147         proc_setas(old_as);
148         as_activate();
149         as_destroy(new_as);
150         kfree(arg_offsets);
151         kfree(arg_buffer);
152         kfree(kern_prog);
153         return ENOMEM;
154     }
155     for (int i = 0; i < argc; i++){
156         char *curr_str = arg_buffer + arg_offsets[i];
157         int len = strlen(curr_str) + 1;
158         stackptr -= len;
159         result = copyout(curr_str, (userptr_t)
160         stackptr, len);
161         if (result){
162             proc_setas(old_as);
163             as_activate();
164             as_destroy(new_as);
165             kfree(arg_offsets);
166             kfree(arg_buffer);
167             kfree(kern_prog);

```

```

162         kfree(string_addr);
163         return result;
164     }
165     string_addr[i] = (userptr_t) stackptr;
166 }
167
168     stackptr -= (stackptr % 8);
169     int pointer_array_size = (argc + 1) * sizeof(
170     userptr_t);
171     if ((stackptr - pointer_array_size) % 8 != 0)
172     stackptr -= 4;
173
174     char *null_ptr = NULL;
175     stackptr -= sizeof(userptr_t);
176     result = copyout(&null_ptr, (userptr_t) stackptr,
177     sizeof(char *));
178     if (result){
179         proc_setas(old_as);
180         as_activate();
181         as_destroy(new_as);
182         kfree(arg_offsets);
183         kfree(arg_buffer);
184         kfree(kern_prog);
185         kfree(string_addr);
186         return result;
187     }
188     for (int i = argc-1; i >= 0; i--){
189         stackptr -= 4;
190         result = copyout(&string_addr[i], (userptr_t)
191         stackptr, sizeof(userptr_t));
192         if (result){
193             proc_setas(old_as);
194             as_activate();
195             as_destroy(new_as);
196             kfree(arg_offsets);
197             kfree(arg_buffer);
198             kfree(kern_prog);
199             kfree(string_addr);
200             return result;
201         }
202     }
203
204     kfree(arg_offsets);
205     kfree(arg_buffer);
206     kfree(kern_prog);
207     kfree(string_addr);

```

```

204
205     // destruction of the old as
206     if (old_as != NULL) as_destroy(old_as);
207
208     /* Warp to user mode. */
209     enter_new_process(argc /*argc*/, (userptr_t)
210                     stackptr /*userspace addr of argv*/,
211                     NULL /*userspace addr of environment*/,
212                     stackptr, entrypoint);
213
214     /* enter_new_process does not return. */
215     panic("enter_new_process returned\n");
216     return EINVAL;
217 #else
218     (void) program;
219     (void) args;
220     return ENOSYS;
221 #endif
222 }

```

4.5.2 Description

The `sys_execv()` system call is the main function used to run a program from the command line (and not from the OS161 menu), that is, without calling `runprogram()`. To implement it, we had to be aware of every detail it required.

The most important one was copying the program name and the arguments from user space to kernel space. This is because, when we create the new address space and switch to it, all of the old user pointers are no longer visible, so if we want to insert the arguments into the stack, we have to keep them in some kernel variables. For the program name it is easy; things get trickier for the arguments.

First, we had to count how many arguments there are, in order to save the value into `argc`. We then used two buffers: `arg_buffer`, which contains all the arguments in one string, and `arg_offset`, which contains, for each element at index `i`, the offset to use in `arg_buffer` to read argument `i`. After opening the file, creating the address space and switching into it, and loading the executable, we create the stack, where we have to follow the calling conventions. We start by inserting into the stack the strings representing the arguments, saving for each one its address in the stack as a pointer. After this, we align the address and create space for the pointers to the arguments, then insert them, terminating with a NULL pointer. It is important to note that the copy is performed from `argc-1` down to 0, since the stack grows downward.

Once this is done, the stack is ready, with all the pointers to the arguments correctly positioned and terminating with a NULL pointer, as required by the POSIX standard. All copy operations are safely delegated to the system

functions `copyout()`, `copyin()`, and `copyinstr()`, so any errors are simply propagated from these functions to the higher-level ones.

If there were no errors, we can safely call `enter_new_process()` to start the new process, passing the number of arguments, the pointer to the stack, and the entrypoint. If this function returns, which it should not, it results in an `EINVAL` error.

5 Conclusions

After all these extension of the base structure of the OS161 system, it is perfectly feasible for multiprogramming. Let's see an hypothetical example of flow execution to see what the system is now capable of. We want to run a program which read the rows of a file and write them on another file with some modifications, the two files are passed by standard input. First, the system boots and the kernel process is created, with the first three standard file descriptors initialized (`stdin`, `stdout`, `stderr`). Then we run the shell, so the kernel process forks (the child will be the shell process), and it waits for the termination of the child. On the shell `stdin`, we run the command "`my_prog text1.txt text2.txt`" and the shell call the `sys_execv()` system call to execute that program. The program will open the two files, read from the first one, do whatever it needs, and write on the second file. After finishing the operation, it will close the two files and terminate. If we exit from the shell, the kernel process will collect its exit status and resume its work.

Every system call checks for every error without crashing and returns it to the system call dispatcher. This way there will be no strange behavior and the system can continue working properly.